Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

232. Implement Queue using Stacks

Solved

Easy | Topics | Companies

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (`push`, `peek`, `pop`, and `empty`).

Implement the `MyQueue` class:

- `void push(int x)` Pushes element x to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns `true` if the queue is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only `push to top`, `peek/pop from top`, `size`, and `is empty` operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

```
Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 1, 1, false]

Explanation
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
```

8.8K | 173 | 73 Online

Code

C++ | Auto

```cpp
class MyQueue {
private:
    stack<int> input;
    stack<int> output;

public:
    MyQueue() {
    }

    void push(int x) {
        input.push(x);
    }

    int pop() {
        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }

        int front = output.top();
        output.pop();

        return front;
    }

    int peek() {
        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }

        return output.top();
    }

    bool empty() {
```

Saved | Ln 1, Col 1

Testcase | Test Result

probal2006
Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features | Orders | My Playgrounds | Settings | Appearance | Sign Out

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

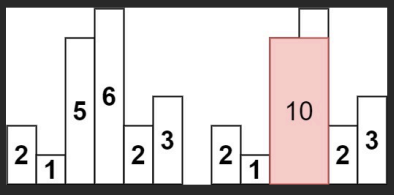
84. Largest Rectangle in Histogram

Solved

Hard | Topics | Companies

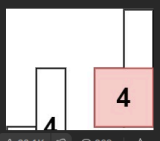
Given an array of integers `heights` representing the histogram's bar height where the width of each bar is `1`, return *the area of the largest rectangle in the histogram.*

Example 1:




```
Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.
```

Example 2:



20.1K | 368 | 124 Online

Code

C++ | Auto

```cpp
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        stack<int> st;
        int maxArea = 0;

        for (int i = 0; i <= heights.size(); i++) {

            int currentHeight;

            if (i == heights.size()) {
                currentHeight = 0;
            } else {
                currentHeight = heights[i];
            }

            while (!st.empty() && heights[st.top()] > currentHeig

                int height = heights[st.top()];
                st.pop();

                int width;

                if (st.empty()) {
                    width = i;
                } else {
                    width = i - st.top() - 1;
                }

                int area = height * width;
                maxArea = max(maxArea, area);
            }

            st.push(i);
        }

        return maxArea;
    }
};
```

Saved | Ln 1, Col 1

Testcase | Test Result

probal2006
Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features | Orders | My Playgrounds | Settings | Appearance | Sign Out